

Python Language Fundamentals – Lecture 2 Summary

Overview

This résumé summarizes the main programming concepts introduced in Lecture 2 of *Programming for Data Science*, covering:

- Language Semantics
- Python Data Types and Structures
- Operators
- Flow Control
- Comprehensions, Slices, and Typecasting

Language Semantics

- Python uses **indentation** (whitespace) instead of braces to define code blocks.
- A **colon** (`:`) denotes the start of an indented block.
- There is no end of statement but you can declare several python statements using

```
a = 5; b = 6; c = 7
# Its the same as
a = 5
b = 6
c = 7
```

- Statements do not need semicolons; they can separate multiple statements on one line.
- **Everything is an object:** numbers, strings, data structures, functions, and modules.
- The `#` symbol introduces **comments**—ignored by the interpreter.

Python Data Types and Structures

Basic Types

You can check the types of the python objects using the `type()` method

- **int** – Integer numbers (5, -10)
- **float** – Real numbers (3.14)
- **complex** – Real + imaginary parts (2 + 3j)
- **str** – Strings of text
- **bool** – True / False values

Collections

- **List** `[]` – Mutable, ordered sequence.
- **Tuple** `()` – Immutable, ordered sequence.
- **Set** `{ }` – Unordered, unique elements.
- **Dictionary** `{key:value}` – Key-value pairs.

Common Methods

- **Lists:** `append()`, `extend()`, `sort()`, `pop()`, `clear()`
- **Tuples:** `count()`, `index()`
- **Sets:** `add()`, `remove()`, `union()`, `intersection()`
- **Dictionaries:** `keys()`, `values()`, `items()`, `update()`

Operators

Arithmetic

`+`, `-`, `*`, `/`, `%`, `**`, `//`

Assignment

`=`, `+=`, `-=`, `*=`, `/=`, `%=`, `//=`, `**=`

Comparison

`==`, `!=`, `>`, `<`, `>=`, `<=`

Logical

`and`, `or`, `not`

Bitwise

`&`, `|`, `^`, `~`, `<<`, `>>`

Identity & Membership

`is`, `is not`, `in`, `not in`

Flow Control

Conditional Statements

- `if`, `elif`, `else` for branching.
- Can be written inline: `print("A") if x > y else print("B")`.
- Combine conditions with `and`, `or`.

Loops

- **while** – executes while condition is true.
- **for** – iterates over elements in a sequence.
- Use **break** to exit loop; **continue** to skip iteration.

Comprehensions

- **List comprehension:** `[x**2 for x in range(10)]`
- **Set comprehension:** `{x for x in range(10)}`
- **Dictionary comprehension:** `{x: x**2 for x in range(5)}`
- **Tuples** don't have comprehensions.

Slices

- General form: `list[i:j:k]`
- **i** – start index, **j** – stop index, **k** – step.
- Negative indices count from the end.

Typecasting

- Manually convert between types: `int()`, `float()`, `str()`, `bool()`
- Example: `int(3.9) → 3`
- Strings to numbers must be properly formatted: `int("42")`